

Detailed Speaker Notes: Decision Trees & Random Forests

Slide 1: Introduction

Introduce yourself and the topic. Explain that Decision Trees and Random Forests are popular supervised learning algorithms. Mention that they are used for both classification and regression tasks. Emphasize that this session will cover both conceptual understanding and practical implementation in Python.

Slide 2: Learning Objectives

Explain what students will learn: 1. The structure and working of Decision Trees. 2. Understanding overfitting and the bias-variance tradeoff. 3. How Random Forests solve the limitations of Decision Trees. 4. Python code demonstrations for both algorithms.

Slide 3: What is a Decision Tree?

A Decision Tree is a supervised learning model that predicts the target variable by learning simple decision rules from features. It splits data recursively based on feature values into branches, leading to a decision (leaf node). Metrics like Gini Index or Entropy are used to determine the best feature to split on. Example: Predicting whether a student passes an exam based on study hours and attendance.

Slide 4: Pros & Cons of Decision Trees

Pros: - Easy to interpret and visualize. - Can handle both numerical and categorical data. - No need for feature scaling. Cons: - Prone to overfitting, especially with deep trees. - Small changes in data can lead to different splits (high variance).

Slide 5: Overfitting & Bias-Variance

Overfitting: Model learns noise in training data, performs poorly on unseen data. Underfitting: Model is too simple to capture patterns. Bias-Variance Tradeoff: We aim for low bias (accurate predictions) and low variance (generalization). Example: A tree with max depth=20 may memorize training data but fail on test data.

Slide 6: What is a Random Forest?

A Random Forest is an ensemble method that builds multiple Decision Trees and combines their predictions. Uses Bagging (Bootstrap Aggregating): Trains each tree on a random subset of the data. Randomness in feature selection ensures trees are decorrelated.
Prediction: Classification → Majority vote, Regression → Average prediction.

Slide 7: Pros & Cons of Random Forests

Pros: - Reduces overfitting compared to a single Decision Tree. - Works well on a wide range of problems. - Robust to noise. Cons: - Less interpretable than a single Decision Tree. - Computationally more expensive, especially with many trees.

Slide 8: Python Example - Decision Tree

Code Explanation: 1. Import DecisionTreeClassifier from sklearn. 2. Set max_depth to prevent overfitting. 3. Fit the model to training data. 4. Use plot_tree to visualize the splits. Tip: Discuss parameter tuning (e.g., criterion='entropy', min_samples_split).

Slide 9: Python Example - Random Forest

Code Explanation: 1. Import RandomForestClassifier. 2. Set n_estimators (number of trees) - common values: 100, 200. 3. Fit the model to training data. 4. Evaluate using score() or cross-validation. Tip: Mention feature_importances_ attribute for feature selection.

Slide 10: Wrap-up

Summarize key points: - Decision Trees: Simple but prone to overfitting. - Random Forests: Ensemble of trees that generalize better. - Both are widely used in finance, healthcare, and recommendation systems. Encourage students to experiment with hyperparameters and datasets.